

Software-Defined OFDM Transceiver: A Physical-Layer Simulation Study

Ojas

June 2026

Contents

1. Abstract & Motivation	1
2. Core Concepts & Terminologies	2
3. Software & Technology Stack	4
4. The Five Phases of Development	5
5. Engineering Difficulties & Trade-offs	8
6. Future Scope	10

An End-to-End Implementation of an 802.11a-Style OFDM Communication System in Python

1. Abstract & Motivation

Orthogonal Frequency-Division Multiplexing (OFDM) is the physical-layer foundation of nearly every modern wireless standard. LTE, 5G NR, Wi-Fi (802.11a/g/n/ac/ax), and DVB-T all rely on the same core mathematics. Most communication systems courses teach this material on paper: students derive the OFDM signal model, study how the Discrete Fourier Transform separates subcarriers, and solve bit error rate (BER) formulas analytically. That kind of fluency matters, but it only goes so far. There's a real gap between being able to derive the BER formula for 16-QAM over an AWGN (noise-only) channel, and actually building a system that proves that formula by generating real bits, corrupting them with simulated noise, and recovering them again.

This project was built specifically to close that gap. The goal wasn't to invent new signal processing theory, but to take the standard OFDM architecture apart piece by piece and rebuild it from scratch in software, starting with a hand-written Fast Fourier Transform (no shortcuts through existing libraries) and ending with a full transmit-channel-receive pipeline that includes realistic channel distortion, channel estimation, equalization, adaptive modulation, and basic error correction.

The principle followed throughout was simple: verify everything, assume nothing. Every component was checked against an independent reference before being trusted as a building block for the next stage. The custom FFT was checked against NumPy's

FFT down to floating-point precision. The OFDM loopback was checked for zero bit errors under ideal conditions. The equalizer's output was checked against the known transmitted constellation. The simulated BER curves were checked against the closed-form theoretical formulas. This habit, treating every claim as something that needs to be demonstrated rather than assumed, runs through all five phases of the project, and it's the single most transferable skill that came out of building it.

The end result is a fully software-based, modular OFDM transceiver that supports four modulation orders (BPSK, QPSK, 16-QAM, 64-QAM), three channel models (AWGN, multipath using ITU-R profiles, and Rayleigh flat fading), two equalizer types (Zero-Forcing and MMSE), basic error correction (Hamming(7,4)), and an adaptive modulation controller, all wrapped in a live, interactive Streamlit dashboard.

2. Core Concepts & Terminologies

This section explains the key communication and signal processing concepts the project is built around, roughly in the order they show up in the system.

2.1 Orthogonal Frequency-Division Multiplexing (OFDM)

OFDM splits one fast data stream into many slower streams, each carried on its own narrow frequency band called a subcarrier. The subcarriers are spaced so precisely that they don't interfere with each other even though they overlap in frequency, a property called orthogonality. This is what lets an Inverse Fast Fourier Transform (IFFT) build all the subcarriers at once on the transmit side, and a regular FFT separate them again on the receive side, without needing a bank of analog filters.

The big practical advantage of OFDM is that it turns a wideband channel, where different frequencies get attenuated differently because of multipath (explained below), into many narrow channels that each look almost flat. Since each subcarrier sees roughly constant attenuation and phase shift, undoing the channel's effect on the receiver side becomes a single complex division per subcarrier, instead of a much more expensive time-domain deconvolution.

2.2 Quadrature Amplitude Modulation (QAM)

QAM encodes bits by placing them on a 2D grid of points called a constellation, where the horizontal axis (I, in-phase) and vertical axis (Q, quadrature) together represent one complex symbol. Higher-order QAM (16-QAM, 64-QAM) packs more bits into each symbol by using more points on a denser grid, which increases throughput but shrinks the spacing between points, making the signal more sensitive to noise. This project uses Gray coding for all four schemes (BPSK, QPSK, 16-QAM, 64-QAM), where neighboring constellation points differ by exactly one bit. That way, if noise pushes a received symbol into a neighboring decision region (the most common kind of error), it only costs one wrong bit instead of two or more.

2.3 Cyclic Prefix (CP)

The cyclic prefix is a short repeated chunk of samples added to the front of every OFDM symbol, copied from the end of that same symbol. It does two things. First, it soaks up the “tail” left behind by multipath echoes so that one symbol doesn’t bleed into the next, which is what prevents Inter-Symbol Interference (explained next). Second, and this is the less obvious part, it changes how the channel’s effect behaves mathematically: instead of linear convolution, the receiver’s FFT window sees circular convolution. That matters because circular convolution in time becomes plain multiplication in frequency, which is exactly why a single division per subcarrier is enough to undo a multipath channel, instead of needing to reverse a full convolution.

2.4 Multipath Fading and Inter-Symbol Interference (ISI)

In any real wireless environment, a transmitted signal reaches the receiver through more than one path: a direct line-of-sight path plus reflections off buildings, terrain, or walls, each arriving with its own delay and attenuation. This is multipath propagation, and it’s modeled here as an FIR (Finite Impulse Response) filter, where each filter tap stands for one path’s relative delay and strength. If left uncorrected, the delayed copies of one symbol smear into the next one and corrupt the bits, a problem called Inter-Symbol Interference. The cyclic prefix is what fixes this in OFDM, as long as it’s longer than the channel’s worst delay spread.

2.5 Rayleigh Flat Fading

Multipath fading is frequency-selective, meaning different subcarriers get hit differently. Flat fading is the opposite case: every subcarrier in one OFDM symbol gets multiplied by the same complex number. This happens physically in places with no single dominant path and lots of similar-strength reflections, where the combined channel effect ends up being a random complex Gaussian value (by the Central Limit Theorem). The magnitude of a complex Gaussian number follows what’s called a Rayleigh distribution, which is where the name comes from. This model is a standard stand-in for dense urban areas or places with no clear line of sight.

2.6 Additive White Gaussian Noise (AWGN)

AWGN is the simplest and most common noise model in communications. Thermal noise at the receiver is treated as a random complex Gaussian signal added on top of the real signal, with equal power spread across all frequencies (hence “white”). Signal-to-noise ratio (SNR) is just signal power divided by noise power, almost always expressed in decibels. AWGN is the channel that the theoretical BER formulas in this report are built around, and it’s used here as the clean baseline that every other channel impairment gets compared against.

2.7 Least-Squares (LS) Channel Estimation

The receiver has no way of knowing in advance exactly how the channel has distorted the signal, so it has to estimate it. This project does that using pilot subcarriers: a

handful of fixed subcarrier positions in every OFDM symbol that carry a value both the transmitter and receiver already agree on ahead of time. At those positions, the channel's effect can be measured directly by dividing the received pilot value by the known transmitted value. This simple division is called the Least-Squares estimate, the most basic estimator there is, and it needs no prior assumptions about the channel's statistics. Since pilots only occupy a few subcarriers, the channel at every other (data) subcarrier is filled in by interpolating between the pilot estimates.

2.8 Zero-Forcing (ZF) and MMSE Equalization

Once the channel is estimated at every subcarrier, the receiver needs to undo its effect, a step called equalization. The simplest method, Zero-Forcing, just divides the received value by the estimated channel gain, which perfectly cancels the channel under ideal, noise-free conditions. The catch is what happens when a subcarrier's channel gain is very small (a deep fade): dividing by a tiny number blows up whatever noise is sitting on that subcarrier. The MMSE (Minimum Mean Square Error) equalizer fixes this by adding a small noise-aware correction term to the division, accepting a bit of leftover channel distortion in exchange for far less noise amplification. That makes MMSE the safer choice at low SNR or in deep fades.

2.9 Forward Error Correction (FEC) and Hamming Codes

Forward Error Correction adds extra, structured bits to the data so the receiver can catch, and sometimes fix, bit errors without needing to ask for a retransmission. The Hamming(7,4) code used here takes 4 data bits and adds 3 parity bits to make a 7-bit codeword, which can correct any single bit error inside that block. The benefit a code provides is called coding gain: how much less SNR you need to hit the same target BER compared to sending the data uncoded. This benefit isn't free or automatic though. At low enough SNR, more than one bit per block gets flipped often enough that a code built to fix only single-bit errors can actually make things worse than sending nothing extra at all. That threshold behavior is covered in detail in Section 5.

3. Software & Technology Stack

Layer	Tool	Role in this project
DSP math engine	NumPy	Vectorized complex-number math, FFT/IFFT reference, array-based signal handling across the whole TX/RX chain

Layer	Tool	Role in this project
Numerical methods	SciPy	<code>scipy.special.erfc</code> for the theoretical BER curves; <code>scipy.signal.welch</code> for power spectral density estimation
Core language	Python 3	All the system logic (modulators, demodulators, channel models, estimators, equalizers) written as modular classes
Static visualization	Matplotlib	Constellation diagrams, BER-vs-SNR curves, channel impulse/frequency response plots, spectral density plots
Interactive frontend	Streamlit	Live, browser-based dashboard with adjustable modulation, channel, SNR, and equalizer controls, plus real-time constellation and BER feedback
Development environment	Jupyter Lab	Phase-by-phase development and inline testing, before everything was pulled together into one shared library file

No ready-made communications libraries (MATLAB's Communications Toolbox, GNU Radio, etc.) were used anywhere in the core signal chain. Every block, from the FFT butterfly to the LS channel estimator, was written by hand. That was deliberate: the point was to show understanding of how these algorithms actually work, not to call something pre-built.

4. The Five Phases of Development

The project was built in five phases, one after another, with an explicit check at the end of each before moving on to the next. This section covers the goal, the method, and the result for each phase.

Phase 1: Custom FFT/IFFT Architecture & Validation

Objective: Build and verify the algorithm everything else in this project depends on, before trusting it for anything further.

A recursive radix-2 Cooley-Tukey FFT was written in plain Python using only the built-in `cmath` module, with no NumPy involved in the actual computation. This was deliberate, to force a real, line-by-line understanding of the divide-and-conquer recursion and the butterfly combine step. The Inverse FFT came almost for free after that, by using the conjugate-symmetry trick between the forward and inverse DFT formulas ($\text{IFFT}(X) = \text{conj}(\text{FFT}(\text{conj}(X))) / N$), which meant a second butterfly algorithm never had to be written.

Validation: The custom FFT was tested against `numpy.fft.fft` for signals of length 8, 64, and 256, using `np.allclose`. All three sizes matched to within floating-point rounding error (around $1e-14$). A full FFT-then-IFFT round trip on a two-tone sine wave reconstructed the original signal exactly.

Outcome: A verified FFT/IFFT pair, plus a magnitude/phase plotting function, ready to act as the core transform for the OFDM modulator and demodulator in Phase 2.

Phase 2: Core OFDM Pipeline

Objective: Build the full transmit and receive chain: QAM mapping, subcarrier assignment, the OFDM modulator, and the OFDM demodulator.

A `QAMMapper` class was built to support BPSK, QPSK, 16-QAM, and 64-QAM, with Gray-coded bit-to-symbol mapping written from scratch (converting Gray code to a regular binary index, then feeding that into a normalized PAM constellation), keeping average symbol energy equal across all four schemes so SNR comparisons stay fair. An `OFDMConfig` class set up the system's subcarrier layout, modeled on 802.11a: 64 total FFT bins, 52 active subcarriers (48 carrying data, 4 carrying pilots), and a 16-sample cyclic prefix.

The `OFDMModulator` handles the full transmit side: bits get mapped to QAM symbols, grouped into blocks, placed onto their assigned data and pilot subcarrier positions in the frequency domain, converted to the time domain with an IFFT, and given a cyclic prefix. The `OFDMDemodulator` undoes every one of those steps in reverse.

Validation: A full loopback test, bits straight through the modulator and into the demodulator with no channel in between, was run for all four modulation schemes. All four came back with zero bit errors, confirming the entire bit-to-waveform-to-bit chain was correct before any channel realism was added.

Outcome: A complete, verified OFDM transmit/receive pipeline, with constellation diagrams and a full subcarrier/time-domain/spectrum plot confirming correct pilot placement, cyclic prefix insertion, and overall signal shape.

Phase 3: Channel Modeling & Equalization

Objective: Introduce realistic channel distortion, then recover from it using estimation and equalization instead of assuming an ideal channel.

Three channel models were built. `AWGNChannel` adds complex Gaussian noise scaled to hit a precise SNR in dB. `MultipathChannel` models frequency-selective fading as an FIR filter with tap delays and power levels taken from ITU-R Pedestrian A/B and Vehicular A profiles, with a random phase on each tap to represent realistic reflection geometry. `RayleighFlatFadingChannel` models block fading by drawing a fresh complex Gaussian value for every OFDM symbol.

To recover from these, an `LSChannelEstimator` computes the channel gain at the pilot positions by direct division, then interpolates between them (carefully handling how the FFT wraps negative frequencies around to the end of the array) to estimate the channel at every data subcarrier. A `FrequencyDomainEqualizer` class then applies either Zero-Forcing or MMSE correction.

Validation: The channel's impulse response and frequency response were plotted straight from the generated tap profile, showing the expected power decay over time and the expected ripple across frequency. Per-subcarrier SNR after equalization was computed and plotted, showing directly where channel nulls still hurt SNR even after correction. Constellation diagrams before and after equalization were generated for AWGN, multipath, and Rayleigh channels, with Error Vector Magnitude (a measure of how far received symbols sit from where they should be) computed for each, confirming that equalization turns a rotated or noise-crushed cloud of points back into a recognizable constellation grid.

Outcome: A working receiver that can estimate an unknown channel from its pilots and correct for it, confirmed both visually (constellation recovery) and numerically (BER, EVM, per-subcarrier SNR).

Phase 4: Monte Carlo BER Validation & Adaptive Modulation

Objective: Move past “it worked once” and check whether the system matches theory across a statistically meaningful number of trials, then add a decision layer that picks modulation order based on channel quality.

A Monte Carlo BER simulation was run directly on QAM over AWGN (skipping the OFDM framing overhead here, to keep the modulation-versus-noise relationship clean) for BPSK, QPSK, and 16-QAM, sweeping SNR from 0 to 30 dB with 20,000 symbols generated per point, well above the 10,000-symbol target needed for a stable BER estimate down to around $1e-4$. These simulated points were plotted directly against the closed-form Gray-coded M-QAM BER formula (built on the complementary error function, `erfc`).

An adaptive modulation controller was built next: theoretical SNR thresholds were worked out for each scheme at a target BER, sorted from lowest to highest, and used to automatically pick the highest-throughput scheme the current SNR could support, the same basic idea behind link adaptation in real LTE and Wi-Fi systems. Spectral efficiency (bits per symbol) was plotted against achieved BER for each fixed scheme, to show the throughput-versus-robustness tradeoff that adaptive modulation is meant to navigate.

Finally, a Hamming(7,4) encoder and decoder were built using a syndrome lookup table for single-error correction, and the coding gain was measured by comparing

coded versus uncoded BPSK BER across a Monte Carlo sweep of E_b/N_0 .

Outcome: Simulated BER curves that match theory within expected statistical noise, a working adaptive modulation selector, and an honestly measured coding gain for Hamming(7,4) that includes its low-SNR penalty (covered in Section 5).

Phase 5: Interactive UI Integration & RF Spectrum Visualization

Objective: Pull every earlier phase together into one working, demonstrable system, and produce a spectrum plot suitable for a portfolio.

Every class built in Phases 1 through 4 was consolidated into one shared library file. A Streamlit app was then built on top of that library, with modulation scheme, channel type, SNR, equalizer choice, and number of simulated OFDM symbols all exposed as live controls. Every time it runs, the dashboard shows measured BER next to theoretical BER, a before-and-after equalization constellation comparison, the power spectral density of the transmitted signal, and the current operating point marked directly on the relevant theoretical BER-vs-SNR curve.

A separate spectrum plot was also generated using Welch’s method on the OFDM transmit signal, with a simulated carrier frequency offset added to model a realistic imperfection in RF hardware, rendered in a green-on-black spectrum-analyzer style for a cleaner look.

Outcome: A publicly runnable, interactive version of the whole system, where SNR and channel conditions can be changed live and the effect on constellation quality and bit error rate shows up immediately, turning five phases of static plots into one explorable tool.

5. Engineering Difficulties & Trade-offs

A few things in this project looked simple as a formula but turned out to be genuinely tricky to get right in a working simulation. This section covers the ones that mattered most.

5.1 The Crossover Penalty in Block Codes at Low SNR

The most surprising result in this whole project was that Hamming(7,4) coding made the BER worse than sending data uncoded, at low E_b/N_0 , before eventually paying off at higher SNR. This isn’t a simulation mistake. It’s a real, well-known property of any error-correcting code with a fixed correction limit. A Hamming(7,4) block can only fix one bad bit per 7-bit codeword. When the channel is noisy enough that two or more bits get flipped in the same block reasonably often, the decoder’s correction logic actively makes things worse: it confidently “fixes” the wrong bit, turning what would have been one recoverable error into extra, decoder-caused errors. Measuring this properly meant finding the exact SNR where the coded and uncoded curves cross, and only reporting coding gain at SNR points safely past that crossover (for example,

at $\text{BER} = 1\text{e-}4$ or $1\text{e-}5$), rather than measuring it right at the crossover point itself, where the apparent gain can look misleadingly close to zero.

5.2 Interpolating Channel Estimates Across the FFT's Frequency Wrap-Around

One detail that's easy to get wrong: standard FFT libraries (including the one used here) order their output bins as [DC, +1, +2, ..., +N/2-1, -N/2, ..., -1], meaning the negative-frequency half of the spectrum is stored after the positive half, not before it. If you interpolate channel estimates across pilot subcarriers using raw FFT bin numbers without accounting for this, you get a broken interpolation, because a pilot at bin 57 (which is actually subcarrier -7 once you account for the wrap-around) would look far away in frequency from a data subcarrier at bin 5, when physically they're close together. The fix was to convert every bin index to its true signed subcarrier position before interpolating, then convert back. That's what's needed to get a smooth, physically correct channel estimate across the full bandwidth.

5.3 Zero-Forcing's Noise Amplification in Deep Fades

Zero-Forcing equalization is, mathematically, the exact inverse of the channel under noise-free conditions, but it has a real practical weak point: at subcarriers sitting in a deep fade (a channel gain close to zero), dividing the received signal by that tiny number blows up whatever noise is on that subcarrier by a huge factor. This showed up directly in the per-subcarrier SNR analysis in Phase 3, where subcarriers caught in a frequency-domain null had visibly worse effective SNR even after "correct" equalization. The fix is MMSE's regularization term, which essentially asks "is dividing by this estimate actually worth the noise penalty, given what's known about the noise floor?" That trades a small, bounded amount of leftover channel distortion for much less worst-case noise blow-up. One more interesting wrinkle showed up here too: in a pure AWGN channel, where the true channel gain is exactly 1 with no fading at all, Zero-Forcing equalization can very slightly increase measured EVM compared to not equalizing at all, because the equalizer ends up dividing by a noisy estimate of a channel that didn't need fixing in the first place. Small effect, but a real reminder that equalization isn't automatically a good thing. It needs to match the actual channel conditions.

5.4 Correctly Scaling Complex Gaussian Noise to a Target SNR

Generating AWGN that hits an exact target SNR in dB sounds simple but is easy to get subtly wrong. Because the noise is complex (it has both an in-phase and a quadrature part), the total noise power has to be split evenly between the two parts: two independent real Gaussian signals, each with half the target total noise variance, not the full variance applied to each one separately. Getting this wrong by a factor of two quietly shifts every BER curve in the system by 3 dB, which is large enough to look like a real result while actually just being a scaling bug. This was caught by directly measuring the actual realized SNR of the generated noise instead of just trusting the requested value, and confirming it matched across several SNR points before relying on the AWGN channel for anything else.

5.5 Telling Real Statistical Noise Apart from an Actual Bug

Monte Carlo BER simulation forces a judgment call that comes up again and again: when a simulated point sits off the theoretical curve, is that just expected statistical variation, or a real bug? At BER levels around $1e-4$ to $1e-5$, a fixed sample size of tens of thousands of symbols might only produce a handful of actual error events, so the measured BER at that point will jump around visibly just from limited sampling, even with a completely correct implementation. A couple of odd-looking results during development (a coding-gain number that didn't make sense, and an early BER point that sat surprisingly far from theory) both turned out to come not from a bug in the DSP itself, but from interpolation code that assumed the data was sorted in increasing order, an assumption that Monte Carlo sampling noise can quietly break. The fix in both cases was to force the data to be properly monotonic (using a running minimum in the right direction) before doing any interpolation on noisy simulated curves, and separately, to run more Monte Carlo trials specifically at higher SNR points, where fewer real error events naturally occur. Knowing when a system is behaving exactly the way the underlying statistics predict, versus when something is actually broken, is probably the single most useful skill that came out of this project.

6. Future Scope

This project intentionally stayed within pure software and a single, basic error-correcting code, so the core physical-layer chain could be fully verified before adding more complexity on top. Two directions follow naturally from here.

Stronger forward error correction. Hamming(7,4) was chosen as a starting point specifically because its small block size makes the encode/decode logic, and its crossover behavior, easy to reason about directly. The natural next step is to replace it with an LDPC (Low-Density Parity-Check) code, the same family of code used in 5G NR's data channel and Wi-Fi 6, which can deliver several more dB of coding gain through iterative decoding instead of a simple syndrome lookup. A convolutional code with Viterbi decoding is a good stepping stone before LDPC, since it introduces a different decoding approach (trellis-based maximum-likelihood decoding) at a more manageable level of complexity than a full LDPC decoder.

Moving from simulation to real RF hardware. Every channel model in this project (AWGN, multipath, Rayleigh) is a deliberate stand-in for effects that happen unpredictably in the real world. The next logical step is to swap these synthetic channels out for an actual over-the-air link, using a low-cost software-defined radio such as an RTL-SDR (receive-only, enough to capture a transmitted waveform from a second device or a cable loopback) or a PlutoSDR (full transmit and receive on one board). That would mean connecting the existing OFDM modulator's output straight to the SDR's transmit buffer, and the demodulator's input to its receive buffer, letting the real RF environment introduce whatever it introduces, including effects this simulation doesn't model yet, like carrier frequency offset that needs active correction, clock drift, and real receiver noise. Getting that loop working would be the natural capstone to this whole project: taking a system that's been verified mathematically at

every stage in software, and proving it survives actual contact with radio-frequency reality.

This document is a technical summary of an independently developed OFDM physical-layer simulation project, covering a custom FFT/IFFT implementation, OFDM modulation and demodulation, multipath and fading channel modeling, LS channel estimation, ZF/MMSE equalization, Monte Carlo BER analysis, adaptive modulation, Hamming(7,4) error correction, and an interactive Streamlit dashboard.